Development of an Integrated DataOps and MLOps Platform Architecture on Kubernetes Using Open Source Tools

Bryan P. Hutagalung
Information Systems and Technology, School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Bandung, Indonesia
18222130@std.stei.itb.ac.id

***Abstract*—DataOps and MLOps are commonly adopted side by side, yet the tools supporting them remain fragmented and the two disciplines are usually run by separate teams. A team wanting both must either assemble the integration itself or subscribe to managed services with recurring cost and provider lock-in. This paper develops an integrated DataOps and MLOps platform architecture on Kubernetes built from open source tools, guided by Design Science Research Methodology from problem identification through design, GitOps-based implementation, and evaluation. The architecture comprises nine platform layers derived from the convergence of five sources, with components selected through scored comparison. The design joins the two disciplines through a shared feature contract, an open lineage protocol feeding one metadata catalog, and one declarative control plane reconciled from a single Git repository. A crypto asset market use case verifies the platform on a single-node cluster. Evaluation covers 26 requirements, of which two are verified functionally, five are instrumented with their targets left unmeasured, and 19 are verified structurally. The retraining chain fires when injected drift crosses the configured threshold, yielding a Population Stability Index of 18.107 against a threshold of 0.2. A cost comparison across three provisioning options places the open source design below fully managed services over three years under an assumed labour allocation. Load characterisation under real traffic remains open.**

***Index Terms*—DataOps, drift detection, feature store, GitOps, Kubernetes, MLOps, open source**

I. Introduction

DataOps improves the quality and speed of the data flow, while MLOps improves the reliability of the model lifecycle. Teams generally need both, yet in practice the two grow apart, because model code is only a small part of a production machine learning system and most surrounding work is data management, configuration, monitoring, and serving [1]. When the two disciplines are run separately, lineage, drift detection, and feature serving end up inconsistent across the cycle.

Building the combined platform in-house means choosing from a wide field of tools. A systematic mapping of 43 primary studies identifies 35 recurring MLOps architecture components [4], and open source components in this space tend to stand alone rather than form a cohesive framework [5]. Integration work, learning curve, and version alignment therefore burden the adopting team from the start.

The alternative is to subscribe to managed services, which reduces configuration work but introduces recurring cost that is hard to estimate in advance and can become uneconomical as load grows [6], leaving organisations to choose between a long build and a subscription with limited customisation.

Neither choice removes the integration work between the ingestion, feature, training, and serving layers. Integration is then reassembled per project, and metadata, identity, and lineage are managed separately, so provenance is hard to trace and feature values can differ between training and serving. Fig. 1 illustrates this fragmented baseline. Surveys of deployment practice report pipeline maintenance and inter-component integration as recurring difficulties [7].

This paper responds with an integrated DataOps and MLOps platform architecture on Kubernetes, built from open source tools. The architecture combines four properties that prior work has addressed separately, namely DataOps and MLOps unified on one Kubernetes control plane, data governance enforced automatically along the pipeline, drift detection that triggers retraining through the same declarative control, and a dual-store feature service under one contract with vector search on a separate index. No individual element is new. The contribution is the combination, which the designs reviewed in Section II do not cover together.

II. Related Work

MLOps is defined as a combination of working culture and software that automates experimentation, training, evaluation, deployment, and monitoring [8]. That definition frames the model side, and DataOps adds ingestion, quality, and governance on the data side [2].

This work sits close to four studies. Burgueño-Romero et al. propose an open source MLOps architecture assembled from components, but a feature service with vector support is not among them [5]. Amou Najafabadi et al. map MLOps architectures across 43 primary studies, but the mapping does not combine DataOps comprehensively [4]. Rodrigues et al. emphasise drift detection and model updating, and their

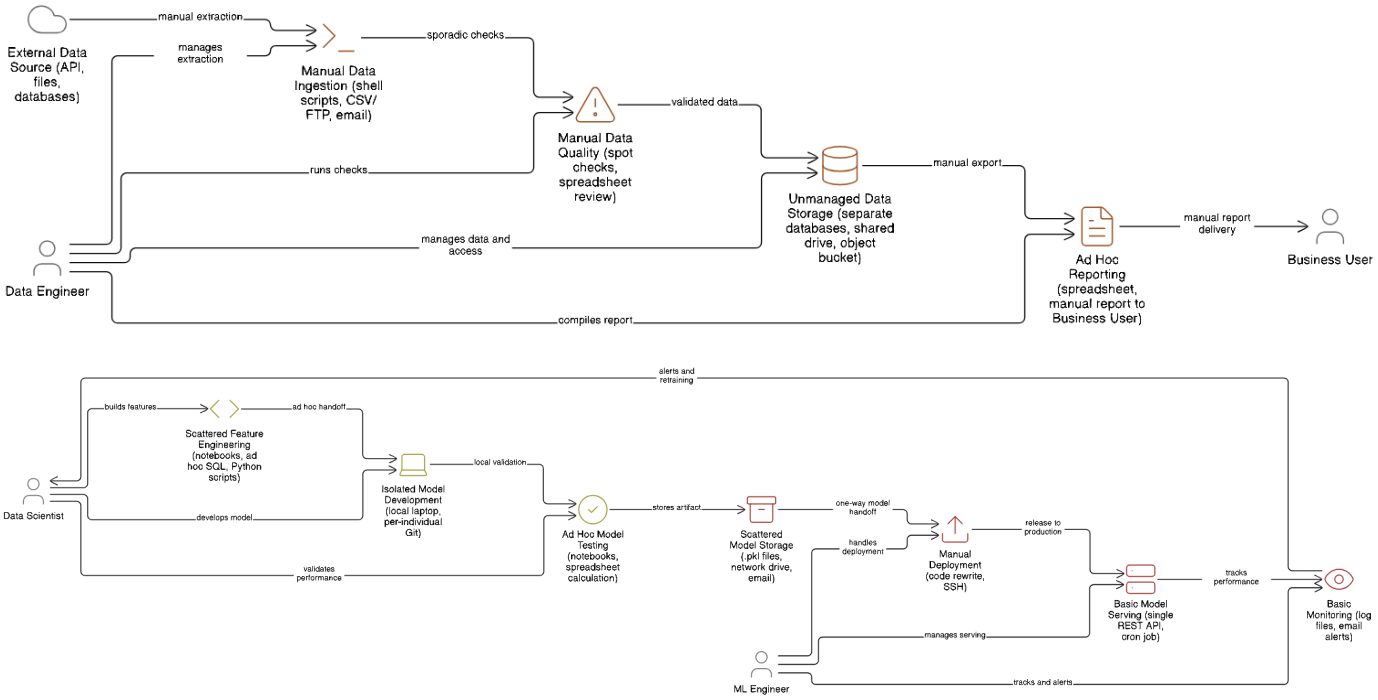

Fig. 1. Fragmented flows before integration. The upper chain shows the data flow on ad hoc tooling at the early stage of the DataOps evolution model [2], and the lower chain shows the manual model flow at the lowest MLOps maturity level [3].

architecture targets near real-time distributed stream learning, though data governance is not enforced as a control plane there [9]. Ahmad et al. discuss MLOps-enabled security strategies without linking them to a governance control plane that enforces access policy across the data and model lifecycle [10].

Because each of these four studies addresses only part of the integration, feature lineage from ingestion through to model serving is hard to trace on one control plane.

A second gap is more specific. Amou Najafabadi et al. observe that an explicit mapping between concrete tools and architecture components is still missing from the MLOps literature [4], and Section IV supplies that mapping for the open source case, with the scoring shown.

III. Method

The work follows Design Science Research Methodology [11]. The main output is a running platform, which is an instantiation artifact in the design science taxonomy [12], so an artifact-oriented framework fits better than a pure software engineering process.

The architecture targets MLOps level 2 on the Google Cloud maturity scale [3], whose seven required components the layers of Section IV-A cover. The fragmented practice of Fig. 1 sits at level 0 on the same scale. Level 2 is what demands the closed loop of continuous integration, continuous delivery, and continuous training that Sections IV-C and IV-D assemble.

The six phases run iteratively. Problem identification produces four problem statements from a study of DataOps, MLOps, and data governance. Objective definition turns each statement into an artifact that can be installed, operated, and audited, giving four objectives.

The first objective unifies ingestion, processing, storage, feature store, training, serving, governance, and observability into one declarative control plane with GitOps practice. The second builds a data governance subsystem providing a metadata catalog, lineage recording, and quality control as shared services along the pipeline. The third implements automated drift detection with a threshold-based retraining policy, and enforces point-in-time correctness across batch and streaming modes through the historical feature join. The fourth develops a dual-store feature service delivering tabular and aggregate features under one API contract, with vector embeddings served on a separate index whose embedding space stays consistent between training and inference.

The objectives are decomposed into 14 functional and 12 nonfunctional requirements, which become the units of evaluation in Section VI. Design and development classify the services named by the sources in Section IV-A into platform layers, select open source components per layer, and express the result as a declarative control plane. Before that, four provisioning alternatives were scored against the same 26 requirements using vendor documentation, and open source on self-managed Kubernetes scored highest on temporal validity, vector support, portability, and extensibility. That desk scoring precedes implementation and is separate from the cost comparison in Section VI-C. Demonstration installs the platform on k3s and drives it with one real domain use case, evaluation examines functional fulfilment, nonfunctional targets, and governance, and communication produces this

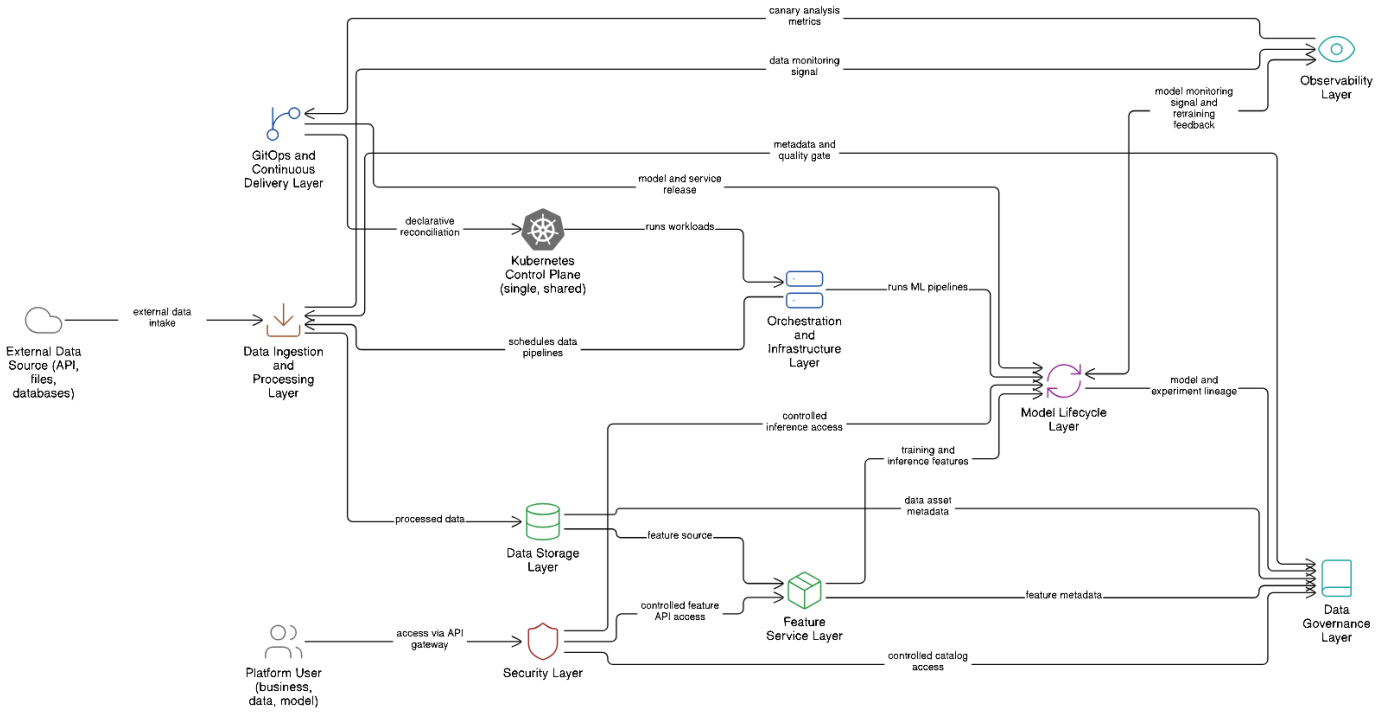

Fig. 2. Integrated service architecture on a shared Kubernetes control plane, drawn without binding to any tool [3].

paper alongside the source repository, available from the author on request.

IV. Architecture

A. Deriving the Platform Layers

The layers are derived from convergence across sources that used different methods, not from majority adoption. On the model side, Kreuzberger et al. name nine MLOps components through mixed-method research combining literature review, tool review, and expert interviews [8]. Amershi et al. arrive at a nine-stage workflow by observing engineering teams at Microsoft, and already separate the data-oriented stages from the model-oriented ones [13]. Starting from different methods, the two name largely the same components. Amou Najafabadi et al. synthesise 35 components from 43 primary studies and report how often each occurs, with model registry at 21 occurrences, ML metadata repository at 15, and feature store at 8 [4]. Google Cloud lists seven components required for MLOps level 2 [3].

No component reaches an absolute majority of the 43 studies, so the counts show recurrence across independent work rather than consensus.

On the data side, Munappy et al. derive an evolution model from ad-hoc analytics to DataOps from an industrial case study, naming data collection, data testing, and data monitoring as recurring practices, and treating controlled access and data-integrity protection as governance [2]. The data side therefore widens the MLOps set rather than duplicating it, and together these five sources form the evidence base.

Consolidating both sides gives the nine layers in Table I. Of the nine, four cover the data domain and five cover the model domain and platform operations. Each row traces to the sources that name its services, and the backing is uneven, from the Model Lifecycle layer named by four sources at once to the Security layer resting mainly on the governance dimension of the DataOps side. Table I leaves that difference visible.

The partitioning sets two boundaries deliberately. Monitoring sits in Observability rather than in Serving, so that measurement does not mix with inference, and drift detection is not a tenth layer but a mechanism crossing layers that is realised as a subsystem.

B. Selecting Components

Every role with more than one viable candidate is scored in a comparison matrix of four criteria chosen for that role, so the criterion set is not fixed across matrices. Maturity, community size, and Kubernetes integration recur most often, while the rest are role-specific, such as batch and streaming capability for processing engines or footprint for distributions. Each criterion scores 0, 1, or 2, giving a maximum total of 8. A score of 0 marks an unmet criterion, 1 partial support with a significant limitation, and 2 full built-in support. Scores are derived from official documentation and each project's maintenance status. In total, 24 matrices cover the roles across the nine layers, with winners in the last column of Table I alongside components adopted without a contest. Of the 24 winners, 19 take a perfect 8. The coarse scale compresses real differences, so the matrices document the selection rather than produce a fine ranking.

TABLE I
PLATFORM LAYERS, THEIR EVIDENCE BASE, AND THE COMPONENTS SELECTED FOR EACH

Layer	Recurring services named by the sources	Sources	Selected open source components
Orchestration and Infrastructure	model training infrastructure, infrastructure and supporting services	[8], [4], [3]	k3s, Horizontal Pod Autoscaler, Vertical Pod Autoscaler recommender, KEDA, Kueue, cert-manager
Data Ingestion and Processing	data collection, data cleaning, data preprocessor, data ingestion	[13], [4], [2]	Kafka with Strimzi, Karapace, Debezium, Flink, Spark, dbt, Airflow, Trino
Data Storage	dataset repository, data storage, data versioning	[4], [2]	MinIO, Apache Iceberg, Lakekeeper, lakeFS, ClickHouse, PostgreSQL, MySQL, OpenSearch
Feature Service	feature store, offline and online serving	[8], [3], [4]	Feast, ClickHouse offline, Valkey online, Qdrant on a separate index outside the Feast contract
Model Lifecycle	model training, model evaluation, model registry, ML metadata store, inference service	[8], [13], [3], [4]	Kubeflow Pipelines, Katib, Kubeflow Trainer, MLflow, KServe, Knative, Argo Workflows
Data Governance	data testing, controlled access, privacy and data integrity protection	[2]	DataHub, OpenLineage, Great Expectations
Observability	monitoring component, model monitoring, data monitoring	[8], [13], [4], [2]	Prometheus, Loki, Grafana Tempo, Grafana, OpenTelemetry, Superset, Evidently, OpenCost
GitOps and Continuous Delivery	CI/CD component, source code repository, deployment services	[8], [3], [4]	Argo CD, Gitea, Tekton, Argo Rollouts
Security	controlled access, privacy and data security compliance, API gateway	[2], [4]	Istio, APISIX, dex, oauth2-proxy, SpiceDB, OpenBao, Kyverno, Falco

The highest total takes the role without removing the other candidates, since a lower-total tool can still be adopted where it is stronger on the criterion that matters. Spark takes the highest total and the large-scale batch role, while Flink, which scores lower overall but higher on event-time processing with exactly-once checkpointing, takes the streaming role, and neither displaces the other. Kubeflow Pipelines takes the training flow, Argo Workflows runs the scheduled control loop, and Airflow orchestrates the batch data pipeline, and the three roles do not overlap.

C. Joining the Two Disciplines

Fig. 2 shows the nine layers as one architecture. DataOps and MLOps meet at three points, where the producing and the consuming side read the same definition rather than two copies.

The first is the feature contract. Feast binds the offline store and the online store to one feature definition, so the pipelines that write features and the training and inference code that read them use the same contract.

The second is open lineage. OpenLineage carries transformation events from the data orchestrator and the batch processing engine into one metadata catalog, which also ingests the feature store and the model registry through its connectors. A training dataset's lineage is then traceable from the same graph that holds the ingestion assets.

The third is the declarative control plane. Argo CD reconciles the manifests of both disciplines from one Git repository, so policy and component versions stay consistent across the two, and it is what makes the drift loop of Section IV-D declarative.

D. Drift Detection and Retraining

Drift detection sits where the two disciplines meet, observed on the data layer while its consequence executes on the model layer. A detector runs as scheduled jobs at several time scales, hourly at the fastest by default, each comparing the current window against the immediately preceding window of the same length. It reads both windows from ClickHouse and computes the Population Stability Index with adaptive binning and the Kolmogorov-Smirnov statistic [14]. Scores and a per-scale retraining flag go to an analytics table in ClickHouse for audit and to Prometheus for alerting.

Every six hours an Argo CronWorkflow queries that table and decides whether to trigger training. The detector's schedules and configuration ship as Git-committed manifests, so a policy change is a commit rather than a live edit. When the flag is set, the workflow calls the training pipeline, which assembles its training set through a point-in-time join and logs the run to the model registry as a new version.

Whether that candidate replaces the running model is decided at canary promotion on live serving metrics, not by a separate comparison before deployment, and promotion is configured to halt and roll back when a metric crosses its threshold.

E. Dual-Store Feature Service

The dual-store design is two stores under one contract. ClickHouse holds offline features for training, Valkey serves online lookups at inference, and both derive their schema from the same Feast definition. Vector embeddings are handled separately, with Qdrant outside Feast as a vector index reached through its native client, because an approximate nearest neighbour load needs memory for the hierarchical navigable small world graph [15] and behaves unlike a key-value point lookup, so each is tuned on its own.

Point-in-time correctness is enforced through the historical feature join, which admits only values whose event timestamp does not exceed each entity row's reference timestamp and still falls inside the feature view's time-to-live. Training therefore sees the values that would have been available at that instant, preventing the optimistic evaluation that leakage produces

TABLE II
VERIFICATION OUTCOME PER OBJECTIVE

Objective	Req.	Func.	Instr.	Struct.
Integrated architecture	12	0	2	10
Data governance	6	1	0	5
Drift and retraining	3	1	0	2
Feature service	5	0	3	2
Total	**26**	**2**	**5**	**19**

[16]. Batch and stream pipelines write to stores registered on the same feature view, so a consumer does not need to know which pipeline produced a value.

V. IMPLEMENTATION

The platform runs on k3s on a single node [17]. The node count is an implementation choice, and moving to a multi-node cluster should change only the Kustomize overlay, though that move has not been performed. Library versions and base images are pinned to the highest stable release as of April 2026 so that testing is reproducible, and a component is intended to be replaceable by an equivalent without changing the design.

Components are grouped one namespace per layer and joined by stable API contracts, which lets network policy default to deny with declared exceptions. Argo CD is the only path by which change reaches the cluster [18], with every component rendered from an ApplicationSet template so adding a component adds no boilerplate. Tekton builds and tests. Istio enforces mutual TLS in strict mode on internal traffic, Kyverno validates at admission, and Falco watches the kernel at runtime. The default is one idle replica per workload, with autoscaling templates prepared but not exercised under load.

Platform code stays domain-agnostic in its own directory, while the domain lives in a separate use-case directory applied through a Kustomize overlay setting only the name prefix, registry mapping, initial replicas, and autoscaling bounds.

The verification use case is crypto asset market analytics on Coinbase data, chosen for three reasons. It exercises all nine layers at once. Crypto markets trade continuously with no closing hours [19], so the ingestion pipeline and stream processing are driven by real event flow around the clock rather than synthetic load on a schedule. Price volatility is high and clustered [20], so feature distributions are expected to shift within hours, which makes the domain suitable for drift work. The drift result in Section VI-B nevertheless comes from a controlled injection, not from natural shift. The use case runs two ingestion pipelines. A WebSocket collector feeds Flink for windowed features, and a REST collector backfills one-minute candles from 1 March 2026.

VI. EVALUATION

A. What Each Claim Rests On

Claims are reported at three tiers naming what was actually observed. A requirement is *structurally verified* when the control plane is formed and reconciled as designed. It is *functionally verified* when a test scenario runs and its output meets the acceptance criterion. It is *instrumented* when the measuring instrument is installed and the flow serves requests, while the quantitative target itself remains unmeasured.

Every scenario ran manually outside the reconciliation path, with the load generator in a separate namespace so that measurement did not disturb the pipeline under test. Table II gives the distribution. The accompanying project report lists all 26 requirements, each with a re-runnable test scenario and a stated acceptance criterion.

B. Results

The integrated architecture and its GitOps path were verified structurally. All components install and reconcile from one Git repository, definition changes converge without drift, and the whole platform was rebuilt on a fresh k3s node with no manual step beyond the initial bootstrap. That reconstruction exercises the entire manifest set rather than a sample, the strongest evidence for the declarative claim.

Governance contributed one of the two functional results. The quality gate rejects schema and range violations before the batch moves downstream. Lineage events from the data orchestrator and the batch processing engine, together with catalog ingestion from the feature store, model registry, warehouse, and broker, assemble into one graph holding 270 datasets, 18 data jobs, and 3 data flows, measuring what the emitters covered rather than completeness across every asset.

Drift detection contributed the other. A controlled 2-sigma injection on one feature produced a Population Stability Index of 18.107 against a 0.2 threshold and a Kolmogorov-Smirnov statistic of 0.815 against a 0.15 threshold, and the scheduled workflow triggered one retraining run. This shows the chain fires end to end, from detection to the retraining trigger, but not detection accuracy against natural distribution shift over time, which needs longer observation on real streams. The point-in-time join named in the same objective was verified structurally, and a probe with future timestamps confirmed it rejects rows that would leak.

The feature service is the weakest of the four. The online, offline, and vector flows are formed and serving requests, but three of its five requirements are instrumented rather than measured. Vector search returned a median of 4.25 ms and a 95th percentile of 5.02 ms over 100 iterations, though the test collection was empty, so those figures describe an installed and responsive flow rather than retrieval quality. Recall was not tested. Equality of feature values between the online and offline stores holds by construction, since materialisation copies from one to the other, but the per-entity comparison confirming it is still manual. Batch and stream consistency was established at the level of feature definitions, not values.

C. Cost

The cost comparison priced three provisioning options against a reference capacity of 16 vCPU, 64 GB memory, and roughly 400 GB storage. Infrastructure figures are public list prices retrieved in June 2026, with two providers priced in

TABLE III
CUMULATIVE COST OF THE PROVISIONING OPTIONS

Option	Three-year total (USD)
Fully managed cloud services	39,543.16
Custom build on self-managed infrastructure	45,461.16
Open source design used here	23,239.22

July where the June archive carried no figure. Table III gives the cumulative totals, with the open source design the cheapest of the three.

The labour allocation behind those totals is an illustrative assumption, not a measurement. The open source option is credited with dropping to a quarter of a full-time engineer after the first month on the argument that reconciliation, unified monitoring, and automated retraining absorb daily operations. The figures therefore show which option is cheaper under that assumption, not what this platform cost to run.

D. Threats to Validity

The test environment is a single node, so horizontal scalability ran as multiple replicas with autoscaling on one machine, and near-linear scaling, p99 under load, peak throughput, and saturation point were not measured. Drift rests on one controlled injection on one feature, and every number comes from a single run with no repetitions and no variance reported. The platform was not benchmarked against a managed stack, so the cost comparison contrasts hypothetical provisioning options rather than measured systems. Domain independence is argued from the separation between platform and use-case code and has not been demonstrated on a second use case.

VII. CONCLUSION

The architecture developed here puts DataOps and MLOps on one Kubernetes control plane using open source tools throughout. The nine layers derive from convergence across sources with different methods, components come from scored comparison, and the disciplines join at the three points of Section IV-C. The value is that integration effort previously repeated per project is consolidated into a single declarative description, so rebuilding no longer depends on one engineer's undocumented knowledge.

The quality gate rejecting bad batches and the automatic firing of the retraining chain are the two results that met an acceptance criterion. The rebuild of the whole platform from one Git source is the strongest of the structural results, and most requirements sit at that tier, which establishes that the control plane forms as designed, not that it performs at production load.

The most immediate next step is load characterisation under real traffic, because behaviour under load is unknown. Drift observation needs longer runs on real streams before detection accuracy can be assessed, and the per-entity comparison of online against offline feature values should be automated so that the equality argued from construction is confirmed by measurement.

REFERENCES

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, Eds. Red Hook, NY: Curran Associates, Inc., 2015, pp. 2503–2511.

[2] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak, "From ad-hoc data analytics to DataOps," in *International Conference on Software and System Processes (ICSSP '20)*. ACM, 2020, pp. 165–174.

[3] Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning," Cloud Architecture Center, 2024. [Online]. Available: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

[4] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, "An analysis of MLOps architectures: A systematic mapping study," in *Software Architecture: 18th European Conference (ECSA 2024)*, ser. LNCS, vol. 14889. Springer, 2024, pp. 69–85.

[5] A. M. Burgueño-Romero, A. Benítez-Hidalgo, C. Barba-González, and J. F. Aldana-Montes, "Toward an open source MLOps architecture," *IEEE Software*, vol. 42, no. 1, pp. 59–64, 2025.

[6] M. Hamza, M. A. Akbar, and R. Capilla, "Understanding cost dynamics of serverless computing: An empirical study," in *Software Business: 14th International Conference (ICSOB 2023)*, ser. LNBIP, vol. 500. Springer, 2024, pp. 456–470.

[7] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.

[8] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.

[9] M. G. Rodrigues, E. K. Viegas, A. O. Santin, and F. Enembreck, "A MLOps architecture for near real-time distributed stream learning operation deployment," *Journal of Network and Computer Applications*, vol. 238, p. 104169, 2025.

[10] T. Ahmad, M. Adnan, S. Rafi, M. A. Akbar, and A. Anwar, "MLOps-enabled security strategies for next-generation operational technologies," in *28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, 2024, pp. 662–670.

[11] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[12] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[13] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 291–300.

[14] D. M. d. Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental Kolmogorov-Smirnov test," in *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1545–1554.

[15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[16] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[17] Cloud Native Computing Foundation, "Kubernetes documentation," 2024. [Online]. Available: https://kubernetes.io/docs/

[18] T. Limoncelli, "GitOps: A path to more self-service IT," *Communications of the ACM*, vol. 61, no. 9, pp. 38–42, 2018.

[19] I. Makarov and A. Schoar, "Trading and arbitrage in cryptocurrency markets," *Journal of Financial Economics*, vol. 135, no. 2, pp. 293–319, 2020.

[20] P. Katsiampa, "Volatility estimation for Bitcoin: A comparison of GARCH models," *Economics Letters*, vol. 158, pp. 3–6, 2017.